

C++ 底层例题代码集（17 例）

18 个文件 · 958 行 · 生成于 2026-10-08 22:55 · 行号对应源文件实际行号

C++ 底层例题代码集 · 目录

17 个例题，每个文件从新的一页开始；讲解见 01/02/03-第X阶段-例题讲解.md。对照用法：先看讲解里的"真实输出"，再翻到这里的代码页，边读边在纸上标出关键行。

第一阶段 · 内存地基（第 1-2 周）

页	文件	主题
2	p1_01_sizeof.cpp	空类、基础类型、对齐、vptr 的字节代价
4	p1_02_layout.cpp	成员顺序与填充、offsetof、逐字节 dump
8	p1_03_stack_heap.cpp	栈向下增长、堆分配、作用域与地址复用
10	p1_04_vptr.cpp	vptr 位置、虚表内容、绕过语法直接按表调用
12	p1_05_polymorphism.cpp	虚函数 vs 非虚函数、两张虚表、虚析构

第二阶段 · 容器与所有权（第 3-6 周）

页	文件	主题
13	p2_01_string.cpp	SSO 边界、容量跳变、reserve 的效果
14	p2_02_vector.cpp	扩容序列、移动而非拷贝、迭代器失效
16	p2_03_raii.cpp	逆序析构、异常路径、禁止拷贝只允许移动
18	p2_04_unique_ptr.cpp	所有权转移、传参三态、自定义删除器
20	p2_05_shared_ptr.cpp	引用计数、循环引用泄漏、weak_ptr 破坏

第三阶段 · 边界与排坑（第 7-12 周）

页	文件	主题
22	p3_01_multi_inherit.cpp	两个 vptr、指针偏移、三种 cast 的区别
24	p3_02_virtual_inherit.cpp	菱形继承两份数据、虚继承的 vbptr
26	p3_03_overflow.cpp	堆越界写 (ASan: heap-buffer-overflow)
27	p3_04_use_after_free.cpp	释放后使用 (ASan: heap-use-after-free)
28	p3_05_double_free.cpp	重复释放 (ASan: attempting double-free)
29	p3_06_project.cpp	综合项目：多态 + 容器 + 智能指针 + 文件
32	p3_07_diagnose.cpp	排坑四步法：切片、悬空引用、容器扩容

三个"故意犯错"的例子请用 clang++ -std=c++17 -g -O0 -fsanitize=address 编译，macOS 上不要加 detect_leaks=1 (会报 not supported)，用ASAN_OPTIONS=abort_on_error=0 运行。

```
1 // 例题 1-1: 空类、基础类型、对齐、vptr 到底各占多少字节
2 // 观察点: sizeof 是"类型合同", 不是"实际用了多少内存"
3 #include <cstdlib>//这个库提供了 size_t 类型
4 #include <cstdio>//这个库提供了 printf 函数
5
6 struct Empty {}; // 空类: 编译器必须给 1 字节占位
7 struct OneChar { char c; };
8 struct CharInt { char c; int i; }; // 1 + 3 填充 + 4 = 8
9 struct IntChar { int i; char c; }; // 4 + 1 + 3 尾部填充 = 8
10 struct HasFunction { void f(); }; // 非虚成员函数不占对象内存
11 struct HasVirtual { virtual void f(); };
12
13 int main() {
14     printf("sizeof(Empty) = %zu\n", sizeof(Empty)); //z: 表示参数类型是
    size_t
15     printf("sizeof(OneChar) = %zu\n", sizeof(OneChar)); //u: 表示按无符号十
    进制整数输出
16     printf("sizeof(CharInt) = %zu (char 后补 3 字节)\n",
    sizeof(CharInt));
17     printf("sizeof(IntChar) = %zu (尾部再补 3 字节)\n",
    sizeof(IntChar));
18     printf("sizeof(HasFunction) = %zu (成员函数不占对象内存)\n",
    sizeof(HasFunction));
19     printf("sizeof(HasVirtual) = %zu (只有一个 vptr)\n",
    sizeof(HasVirtual));
20     printf("\n-- 基础类型 (%zu 位平台) --\n", sizeof(void *) * 8);
21     printf("bool=%zu char=%zu short=%zu int=%zu long=%zu long long=%zu\n",
22         sizeof(bool), sizeof(char), sizeof(short), sizeof(int),
    sizeof(long),
23         sizeof(long long));
24     printf("float=%zu double=%zu long double=%zu\n",
25         sizeof(float), sizeof(double), sizeof(long double));
26     printf("int*=%zu void*=%zu 函数指针=%zu\n",
27         sizeof(int *), sizeof(void *), sizeof(void (*)()));
28     Empty a, b;
29     printf("\n两个空类对象: %p 与 %p, 相差 %ld 字节\n",
30         (void *)&a, (void *)&b, (long)((char *)&b - (char *)&a));
31     return 0;
32 }
```

p1_01_sizeof.cpp · 内存结构图

按本机实测绘制 (Apple clang 21 / arm64 / macOS) : 1 格 = 1 字节, 数据成员、填充、vptr 分色。

数据成员 对齐填充 vptr (虚表指针) 占位字节

Empty 空类: 为什么不是 0 字节 sizeof = 1 字节

占位字节

0

C++ 要求每个对象都有唯一地址, 空类也必须占 1 字节。

OneChar 只含一个 char

sizeof = 1 字节

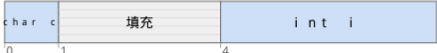
char c

0

char 对齐 = 1, 所以大小就是 1, 没有任何填充。

CharInt char 在前、int 在后

sizeof = 8 字节



int 必须落在 4 的倍数上: char 占偏移 0, int 只能从偏移 4 开始, 中间 3 字节变成填充; 整体按 4 对齐。

IntChar int 在前、char 在后

sizeof = 8 字节



int 在偏移 0、char 在偏移 4; 整体必须按 4 对齐, 尾部又补 3 字节 —— 大小是 8, 不是 5。

HasFunction 只有非虚成员函数

sizeof = 1 字节

占位字节 (无数据成员)

0

成员函数不占对象内存: 它编译成普通函数, 调用时额外把 this 传进去, 所以 sizeof 仍是 1。

HasVirtual 有一个虚函数

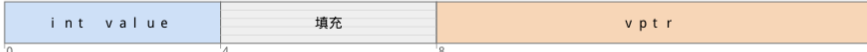
sizeof = 8 字节



出现虚函数后对象里多出一个 vptr (64 位下 8 字节), 它指向该类唯一的虚表。

WithVirtual int + 两个虚函数 (例题 1-4)

sizeof = 16 字节



数据成员先按对齐落位 (int 在 0, 补到偏移 8), 再放 vptr —— 16 字节。

LooseLayout char → double → int (例题 1-2)

sizeof = 24 字节



double 要 8 对齐, char 后面跳 7 字节; int 之后整体再补到 8 的倍数 —— 24 字节。

TightLayout double → int → char (例题 1-2)

sizeof = 16 字节



同样的三个成员, 只改了顺序: 16 字节, 比上面省 8 字节 —— 成员排序也是优化。

三条规则

1. 成员按自身对齐要求落位: char = 1、int = 4、double = 8、指针 = 8; 放不下就往后挪, 空洞变成填充。
2. 结构体整体按最大成员对齐; 尾部常要补一段, 保证数组里每个元素都对齐 (IntChar 是 8 的原因)。
3. 空类占 1 字节占位; 有虚函数多加 8 字节 vptr —— 这两条与「有几个成员」无关。

```
1 // 例题 1-2: 成员顺序决定内存布局; offsetof 与字节 dump
2 // 观察点: 对齐填充出现在哪里、为什么调整顺序能省内存
3 #include <cstddef>
4 #include <cstdio>
5 #include <string>
6
7 struct LooseLayout { // 顺序不好: char → double → int
8     char c;
9     double d;
10    int i;
11 };
12
13 struct TightLayout { // 顺序调整后能省 8 字节
14     double d;
15     int i;
16     char c;
17 };
18
19 struct WithString { // 含库类型: std::string 在 libc++ 里是 24 字节
20     int i;
21     std::string s;
22     char c;
23 };
24
25 template <typename T>
26 static void dump_bytes(const T &obj, const char *name) {
27     const unsigned char *p = reinterpret_cast<const unsigned char *>(&obj);
28     printf("%-12s (%2zu 字节): ", name, sizeof(T));
29     for (size_t i = 0; i < sizeof(T); ++i) printf("%02x ", p[i]);
30     printf("\n");
31 }
32
33 int main() {
34     printf("sizeof(LooseLayout) = %zu\n", sizeof(LooseLayout));
35     printf("  offsetof c=%zu d=%zu i=%zu\n",
36           offsetof(LooseLayout, c), offsetof(LooseLayout, d),
37           offsetof(LooseLayout, i));
38     printf("sizeof(TightLayout) = %zu\n", sizeof(TightLayout));
39     printf("  offsetof d=%zu i=%zu c=%zu\n",
40           offsetof(TightLayout, d), offsetof(TightLayout, i),
41           offsetof(TightLayout, c));
42     printf("同样的三个成员, 顺序不同差了 %zu 字节.\n\n",
43           sizeof(LooseLayout) - sizeof(TightLayout));
44
45     LooseLayout loose{};
46     loose.c = 0xAA;
47     loose.d = 1.0;
48     loose.i = 0x11223344;
49     dump_bytes(loose, "LooseLayout");
50     printf("  → 0xAA 后面的 7 个 0 是填充, int 落在偏移 16.\n");
```

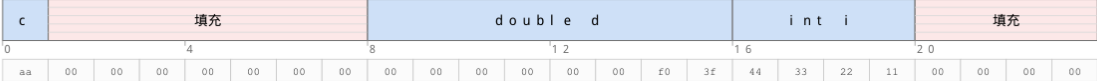
```
50     TightLayout tight{};
51     tight.d = 1.0;
52     tight.i = 0x11223344;
53     tight.c = 0xAA;
54     dump_bytes(tight, "TightLayout");
55     printf("    → 填充被挤到尾部, 只剩 3 字节。\\n\\n");
56
57     printf("sizeof(std::string) = %zu (libc++ 固定 24 字节) \\n",
sizeof(std::string));
58     printf("sizeof(WithString) = %zu  offsetof(s)=%zu  offsetof(c)=%zu\\n",
59             sizeof(WithString), offsetof(WithString, s), offsetof(WithString,
c));
60     WithString ws{};
61     ws.i = 1;
62     ws.s = "abc";
63     ws.c = 'x';
64     dump_bytes(ws, "WithString");
65     printf("    → 前 4 字节是 i, 3 字节填充, 再往后 24 字节是 string 本体。\\n");
66     return 0;
67 }
```

p1_02_layout.cpp · 内存结构图

按本机实测绘制 (Apple clang 21 / arm64 / macOS) : 1 格 = 1 字节。上排是字段归属，中间是偏移标尺，下排是实测 dump 出的原始字节。

数据成员 填充 (内容未定义) 库类型内部 标志位 / 指针字段

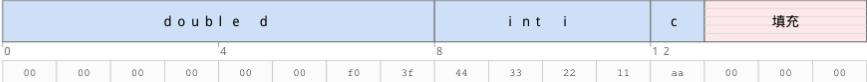
LooseLayout char → double → int (顺序不好) sizeof = 24 字节



下排 = 本例 dump 出的原始字节 (小端; ?? = 填充内容未定义, 实测每次运行都不同)

- double 要求 8 对齐: char 占偏移 0, double 只能从偏移 8 开始, 中间 7 字节变成填充; int 落在偏移 16, 尾部再补 4 → 24。
- 字节全对上: aa 是 char; f0 3f 是 double 1.0 的高位 (小端); 44 33 22 11 是 int 0x11223344。
- 这里的填充显示为 00, 是因为 LooseLayout 是平凡类型, 'l {}' 值初始化会连填充位一起清零; 换成含非平凡成员的 With String, {} 就不再清零填充位 —— 下一页就是反例。

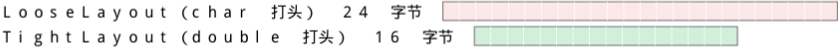
TightLayout double → int → char (调整后) sizeof = 16 字节



下排 = 本例 dump 出的原始字节 (小端; ?? = 填充内容未定义, 实测每次运行都不同)

- 同样三个成员: 8 + 4 + 1 + 3 = 16; 填充被挤到尾部, 只剩 3 字节。
- 规则: 成员按自身对齐落位, 结构体整体按最大成员对齐 —— 把大的放前面、小的挤到最后, 就能把填充集中到尾部一块。

同样的三个成员，只改声明顺序



- 同样三个成员、同样的值, 只改声明顺序: 24 → 16, 省 8 字节 (约 33%)。
- Rust 默认会重排字段来省内存, C++ 不会: 手动排序是 C++ 程序员的基本功。

三条规则

1. 成员按自身对齐要求落位: char = 1、int = 4、double = 8、指针 = 8; 放不下就往后挪, 空洞变成填充。

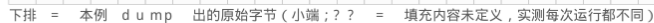
2. 结构体整体按最大成员对齐, 尾部常要补一段, 保证数组里每个元素都对齐 (LooseLayout 是 8 对齐、TightLayout 也是 8 对齐)。

3. 填充属于对象的一部分 (sizeof 算进去), 但它的内容未定义: 不要 memcmp 比较含填充的结构体, 也不要按字节序列化后跨平台传 —— 这正是宿主层与前端交换数据时要先定协议的原因。

With String 把「库类型」也放进来: `std::string` 的本体大小是固定的, 但字符串数据在哪, 取决于长度。

数据成员
 填充 (内容未定义)
 std::string 本体
 指针 / 标志字段

```
WithString int + std::string + char          sizeof = 40 字节
```



4 + 4 + 24 + 1 + 7 = 40。std::string 要 8 对齐,所以 i 之后空出 4 字节;c 之后为对齐到 8 的倍数再补 7 字节。

算出来的是「本体 24 字节」，不是「字符串 3 字节」—— `sizeof` 是类型合同，数组步长、跨语言传参的 ABI 都按它走。

· 偏移 4 . . 7 和 33 . . 39 是本例里唯一没被写过的位置：多次运行值都不一样，所以图上记 ??

`std::string` 本体 · 短串 (SSO) `len ≤ 22` : 数据就放在对象里 `sizeof = 24` 字节



· 实测：字符串本体 24 字节 = 23 字节缓冲 + 1 字节长度；缓冲起始地址就等于对象自身地址，没有任何堆分配。

· libcpp 采用“数据在前”布局：最后 1 字节是长度，len=22 时它等于 0x16；超过 22 字符（len=23）就整体切成长串模式。

`std::string` 本体 · 长串(堆) `len ≥ 23` : 对象里只留指针 `sizeof = 24` 字节



实测 `len=40;size=40(0x28)、capacity=47、data` 在堆上(指针值每次运行都不同,图上记??);容量字段去掉最高位是 `0x30 = 48 = 容量 47 + 结尾 '\0'`。

· 所以「std::string 占 24 字节」这句话在两种模式下都成立，但「字符串数据在哪」完全不同——这就是例题 2-1 的伏笔。

1. 填充字节的内容是未定义的。本机实测: `WithString` 的填充每次运行都不一样; 把缓冲区先填成 `0xAB`、再 `placement new` 构造对象, 构造后填充位置仍是 `ab` —— 证明编译器根本不会去写它。

2. `std::string` 本体固定 24 字节，数据的位置随长度变： ≤ 22 字符就地存放（SSO，零堆分配）， ≥ 23 字符变成「指针 + 大小 + 容量」三个 8 字节字段。

3. WithString 的 40 = 4(int) + 4(填充) + 24(string 本体) + 1(char) + 7(尾部填充), 与例题的 dump 逐字节对得上; 字段值能认出来, 填充位认不出来。

```
1 // 例题 1-3: 栈与堆的本质区别 (地址方向、生命周期、谁负责释放)
2 #include <stdio>
3
4 static int g_counter = 0;
5
6 static void level(int depth) {
7     int local = depth;
8     printf("level(%d): 栈变量 &local = %p\n", depth, (void *)&local);
9     if (depth < 3) level(depth + 1);
10 }
11
12 struct Node {
13     int value;
14     Node *next;
15 };
16
17 int main() {
18     printf("-- 栈: 向低地址方向增长 --\n");
19     level(0);
20     printf("g_counter (全局/静态区) 地址 = %p\n\n", (void *)&g_counter);
21
22     printf("-- 同一函数里的局部变量 --\n");
23     int a = 1, b = 2, c = 3;
24     printf("&a=%p &b=%p &c=%p (地址不保证连续) \n", (void *)&a, (void *)&b,
25 (void *)&c);
26     int arr[4] = {0, 1, 2, 3};
27     for (int i = 0; i < 4; ++i) {
28         printf("arr[%d] 值=%d 地址=%p (同一数组内连续, 步长 %zu) \n",
29             i, arr[i], (void *)&arr[i], sizeof(int));
30     }
31
32     printf("\n-- 堆: 地址由分配器决定, 通常与栈相距很远 --\n");
33     Node *head = nullptr;
34     for (int i = 0; i < 3; ++i) {
35         Node *node = new Node{i, head};
36         printf("new Node{value=%d} → 堆地址 %p\n", i, (void *)node);
37         head = node;
38     }
39     printf("头指针 head 本身在栈上: &head=%p (%zu 字节), 指向堆上的节点\n",
40         (void *)&head, sizeof(head));
41
42     printf("\n-- 生命周期 --\n");
43     {
44         int scoped = 42;
45         printf("作用域内的 scoped 在 %p\n", (void *)&scoped);
46     }
47     int reused = 7;
48     printf("作用域外的 reused 在 %p (地址常被复用, 说明栈空间是重复利用的) \n",
49         (void *)&reused);
50
51     printf("\n-- 释放: 栈自动, 堆手动 --\n");
```

```
51     while (head) {
52         Node *next = head->next;
53         printf("delete 堆节点 %p (值 %d) \n", (void *)head, head->value);
54         delete head;
55         head = next;
56     }
57     return 0;
58 }
```

```

1 // 例题 1-4: 虚函数如何改变对象内存 (vptr 与虚表)
2 // 观察点: vptr 在对象最前面; 虚表是每个类一张全局表, 对象只存一个指针
3 #include <cstdio>
4
5 struct Plain {
6     int value;
7     void f() { printf("Plain::f()\n"); }
8 };
9
10 struct WithVirtual {
11     int value;
12     virtual void f() { printf("WithVirtual::f()\n"); }
13     virtual void g(int x) { printf("WithVirtual::g(%d)\n", x); }
14 };
15
16 struct Derived : WithVirtual {
17     void f() override { printf("Derived::f()\n"); }
18 };
19
20 static void dump_vtable(void *obj, const char *name, int slots) {
21     void **vptr = *reinterpret_cast<void ***>(obj);
22     printf("%-15s vptr=%p\n", name, (void *)vptr);
23     for (int i = 0; i < slots; ++i) printf("    vtable[%d] = %p\n", i,
24     vptr[i]);
25 }
26
27 int main() {
28     printf("sizeof(Plain)          = %zu (int + 填充) \n", sizeof(Plain));
29     printf("sizeof(WithVirtual) = %zu (int 4 + 填充 4 + vptr 8) \n",
30     sizeof(WithVirtual));
31     printf("sizeof(Derived)       = %zu (没加成员, 只是换掉表里的函数) \n\n",
32     sizeof(Derived));
33
34     WithVirtual base;
35     WithVirtual base2;
36     Derived derived;
37     dump_vtable(&base, "WithVirtual", 2);
38     dump_vtable(&base2, "WithVirtual#2", 2);
39     dump_vtable(&derived, "Derived", 2);
40     printf("→ 同类对象共用同一张表; 子类重写后表里的函数地址不同。 \n\n");
41
42     printf("对象前 8 字节的原始值 (就是 vptr) : ");
43     unsigned char *raw = reinterpret_cast<unsigned char *>(&base);
44     for (int i = 0; i < 8; ++i) printf("%02x", raw[i]);
45     printf("\n\n--- 绕过语法, 直接按虚表调用 (多态的机械本质) ---\n");
46     void **vptr = *reinterpret_cast<void ***>(&base);
47     auto f0 = reinterpret_cast<void (*)>(WithVirtual *)>(vptr[0]);
48     auto f1 = reinterpret_cast<void (*)>(WithVirtual *, int)>(vptr[1]);
49     f0(&base);
50     f1(&base, 42);
51     return 0;

```



```
1 // 例题 1-5: 多态 = 运行时通过 vptr 查表; 非虚函数 = 编译期按指针类型绑定
2 #include <cstdio>
3
4 struct Base {
5     virtual void speak() { printf("Base::speak()\n"); }
6     void name() { printf("Base::name()    ← 非虚, 看指针的静态类型\n"); }
7     virtual ~Base() { printf("~Base()\n"); }
8 };
9
10 struct Derived : Base {
11     void speak() override { printf("Derived::speak()\n"); }
12     void name() { printf("Derived::name() ← 非虚, 看指针的静态类型\n"); }
13     virtual void extra() { printf("Derived::extra() ← 新增虚函数, 追加在表尾\n"); }
14     ~Derived() override { printf("~Derived()\n"); }
15 };
16
17 int main() {
18     Derived d;
19     Base *pb = &d;
20     Derived *pd = &d;
21     printf("&d=%p  pb=%p  pd=%p (单继承下三者地址相同) \n", (void *)&d, (void *)pb, (void *)pd);
22
23     printf("\n-- 虚函数: 运行时按对象真实类型查表 --\n");
24     pb->speak();
25     pd->speak();
26
27     printf("\n-- 非虚函数: 编译期按指针类型绑定 --\n");
28     pb->name();
29     pd->name();
30
31     printf("\n-- 打印两张不同的虚表 --\n");
32     Base fake;
33     void **base_vptr = *reinterpret_cast<void ***>(&fake);
34     void **derived_vptr = *reinterpret_cast<void ***>(pd);
35     printf("Base 对象 vptr=%p, Derived 对象 vptr=%p\n", (void *)base_vptr, (void *)derived_vptr);
36     printf("speak 槽位: Base=%p  Derived=%p\n", base_vptr[0], derived_vptr[0]);
37
38     printf("\n-- 虚析构: 用基类指针 delete 必须靠它 --\n");
39     Base *heap = new Derived();
40     delete heap;
41     printf("(~Base 不是 virtual 时, 上面只会打印 ~Base, Derived 的成员就泄漏了)\n");
42     return 0;
43 }
```

```
1 // 例题 2-1: std::string 的 SSO、容量与扩容
2 // 观察点: 小字符串的缓冲就在对象内部; 越界那一刻数据会搬到堆上
3 #include <cstdint>
4 #include <cstdio>
5 #include <string>
6
7 static void show(const std::string &s, const char *label) {
8     const char *data = s.data();
9     const char *self = reinterpret_cast<const char *>(&s);
10    bool inside = data >= self && data < self + sizeof(s);
11    printf("%-10s size=%2zu capacity=%2zu data=%p SSO=%s\n",
12          label, s.size(), s.capacity(), (const void *)data, inside ? "内部
    缓冲" : "堆上");
13 }
14
15 int main() {
16     printf("sizeof(std::string) = %zu (libc++: 指针/长度/容量 24 字节, SSO 缓冲嵌
    在里面) \n\n",
17           sizeof(std::string));
18
19     std::string shortStr = "hi";
20     std::string mediumStr = "01234567890123456789"; // 20 字符
21     std::string longStr = "0123456789012345678901234567890"; // 31 字符
22     show(shortStr, "短字符串");
23     show(mediumStr, "20 字符");
24     show(longStr, "31 字符");
25     printf("→ libc++ 的 SSO 上限是 22 个字符: 不超过就放在对象内部, 不碰堆。 \n\n");
26
27     printf("--- 逐步增长, 看容量怎么跳 --\n");
28     std::string s;
29     for (int i = 0; i < 40; ++i) {
30         s += 'x';
31         if (i < 4 || i == 20 || i == 21 || i == 22 || i == 23 || i == 39) {
32             printf("长度 %2d: capacity=%2zu data=%p%s\n", (int)s.size(),
33                   s.capacity(),
34                   (const void *)s.data(),
35                   s.capacity() > 22 ? " ← 已经在堆上了" : "");
36         }
37     }
38     printf("\n--- 扩容会重新分配 + 拷贝, 预留容量可避免 --\n");
39     std::string r;
40     r.reserve(64);
41     const char *before = r.data();
42     for (int i = 0; i < 40; ++i) r += 'y';
43     printf("reserve(64) 后: capacity=%zu 追加 40 次, data 地址%s\n",
44           r.capacity(), r.data() == before ? "没变 (一次分配搞定)" : "变了 (发
    生了重分配)");
45     return 0;
46 }
```

```
1 // 例题 2-2: vector 的连续内存、扩容倍数、迭代器/指针失效
2 #include <cstdio>
3 #include <vector>
4
5 struct Tracked {
6     int value;
7     static int alive;
8     static int copies;
9     static int moves;
10    explicit Tracked(int v) : value(v) { ++alive; }
11    Tracked(const Tracked &other) : value(other.value) { ++alive; ++copies; }
12    Tracked(Tracked &&other) noexcept : value(other.value) { ++alive; ++moves; }
13    ~Tracked() { --alive; }
14 };
15 int Tracked::alive = 0;
16 int Tracked::copies = 0;
17 int Tracked::moves = 0;
18
19 int main() {
20     printf("--- 基本类型: 扩容时 size/capacity/地址怎么变 --\n");
21     std::vector<int> v;
22     const void *last = nullptr;
23     for (int i = 0; i < 17; ++i) {
24         v.push_back(i);
25         if (v.data() != last) {
26             printf("push_back(%2d) → size=%2zu capacity=%2zu data=%p ← 发生\n",
27                 i, v.size(), v.capacity(), (const void *)v.data());
28             last = v.data();
29         }
30     }
31     printf("容量序列大致是 1,2,4,8,16,32: libc++ 按 2 倍增长, 均摊 O(1)。\\n\\n");
32
33     printf("--- 元素是自定义类型: 搬家的代价能被看见 --\n");
34     {
35         std::vector<Tracked> items;
36         for (int i = 0; i < 8; ++i) {
37             items.emplace_back(i);
38         }
39         printf("插入 8 个元素后: 存活=%d 拷贝=%d 移动=%d\\n",
40             Tracked::alive, Tracked::copies, Tracked::moves);
41         printf("→ 扩容时用移动 (Tracked 有 noexcept 移动构造), 拷贝次数为 0 才是合格实现。\\n");
42     }
43     printf("vector 离开作用域后: 存活=%d (全部析构, 没有泄漏) \\n\\n",
44         Tracked::alive);
45
46     printf("--- 迭代器/指针失效 --\n");
47     std::vector<int> w{1, 2, 3};
```

```
47     const int *p = w.data();
48     printf("扩容前 data=%p\n", (const void *)p);
49     w.push_back(4);
50     printf("push_back 后 data=%p, 旧指针 %s\n", (const void *)w.data(),
51           p == w.data() ? "仍然有效" : "已悬空 (继续解引用就是 UB, ASan 会抓
到)");
52     w.reserve(64);
53     printf("reserve 之后 data=%p, 之前那个指针同样失效\n", (const void
*)w.data());
54     printf("\n结论: 只要发生重分配, 所有指向元素的指针/引用/迭代器全部作废.\n");
55     return 0;
56 }
```

```
1 // 例题 2-3: RAII —— 把"释放"这件事交给析构函数
2 // 观察点: 无论正常返回、提前 return 还是抛异常, 析构都会执行, 账本一定归零
3 #include <cstddef>
4 #include <cstdio>
5 #include <stdexcept>
6 #include <utility>
7 #include <vector>
8
9 static int g_outstanding = 0; // 当前还没释放的堆分配数 (内存账本)
10
11 class Buffer {
12 public:
13     Buffer(std::size_t n, const char *tag)
14         : tag_(tag), size_(n), data_(new unsigned char[n]) {
15         ++g_outstanding;
16         printf(" 构造 %-8s: 申请 %zu 字节 @ %p (账本=%d) \n", tag_, n, (void
17 *)data_, g_outstanding);
18     }
19     Buffer(const Buffer &) = delete; // 独占资源, 禁止拷贝
20     Buffer &operator=(const Buffer &) = delete;
21     Buffer(Buffer &&other) noexcept // 只允许移动: 转移所有权
22         : tag_(other.tag_), size_(other.size_), data_(other.data_) {
23         other.data_ = nullptr;
24         other.size_ = 0;
25         printf(" 移动 %-8s: 指针移交给新对象 (账本不变=%d) \n", tag_,
26 g_outstanding);
27     }
28     ~Buffer() {
29         if (data_) {
30             delete[] data_;
31             --g_outstanding;
32             printf(" 析构 %-8s: 释放 @ %p (账本=%d) \n", tag_, (void *)data_,
33 g_outstanding);
34         }
35     }
36     const char *tag() const { return tag_; }
37     std::size_t size() const { return size_; }
38
39 private:
40     const char *tag_;
41     std::size_t size_;
42     unsigned char *data_;
43 };
44
45 static void raiiVersion() {
46     printf("[RAII 版本]\n");
47     Buffer first(1024, "first");
48     Buffer second(2048, "second");
49     printf(" → 出作用域时按构造的逆序析构: \n");
50 }
```

```

49 static void rawVersion() {
50     printf("[裸指针版本]\n");
51     unsigned char *raw = new unsigned char[1024];
52     ++g_outstanding;
53     printf("    new 了 1024 字节 @ %p, 但忘了 delete\n", (void *)raw);
54 }
55
56 static void throwVersion() {
57     printf("[中途抛异常]\n");
58     Buffer guard(512, "guard");
59     throw std::runtime_error("模拟失败");
60 }
61
62 int main() {
63     raiiVersion();
64     printf("    账本 = %d\n\n", g_outstanding);
65
66     rawVersion();
67     printf("    账本 = %d ← 这 1 就是泄漏, ASan/LSan 会报 leak\n\n",
68 g_outstanding);
69     g_outstanding = 0; // 演示用: 人为清账, 避免影响后面的输出
70
71     try {
72         throwVersion();
73     } catch (const std::exception &e) {
74         printf("    捕获异常: %s\n\n", e.what());
75     }
76     printf("    账本 = %d (异常路径也被清理干净) \n\n", g_outstanding);
77
78     printf("[把 Buffer 放进 vector: 只移动, 不拷贝]\n");
79     {
80         std::vector<Buffer> pool;
81         pool.reserve(2);
82         pool.emplace_back(128, "pool-1");
83         pool.emplace_back(256, "pool-2");
84         pool.emplace_back(512, "pool-3"); // 触发扩容: 已有元素被移动
85         printf("    vector 内元素数=%zu, 账本=%d\n", pool.size(), g_outstanding);
86     }
87     printf("    vector 销毁后账本 = %d\n", g_outstanding);
88     return 0;
89 }

```

```
1 // 例题 2-4: unique_ptr —— 编译期就禁止"共享", 对齐 Rust 的所有权直觉
2 #include <cstdio>
3 #include <memory>
4 #include <utility>
5 #include <vector>
6
7 struct Resource {
8     int id;
9     explicit Resource(int i) : id(i) { printf(" 构造 Resource(%d)\n", id); }
10    ~Resource() { printf(" 析构 Resource(%d)\n", id); }
11 };
12
13 static void consume(std::unique_ptr<Resource> p) { // 按值接收 = 拿走所有权
14     printf(" consume 里看到的 id=%d\n", p->id);
15 } // 离开函数即析构
16
17 static void observe(const std::unique_ptr<Resource> &p) { // 借用, 不转移
18     printf(" observe 里看到的 id=%d (所有权还在调用方) \n", p->id);
19 }
20
21 int main() {
22     printf("-- 构造与自动释放 --\n");
23     {
24         auto p = std::make_unique<Resource>(1);
25         observe(p);
26         printf(" 离开作用域 → 自动 delete\n");
27     }
28
29     printf("\n-- 移动: 所有权转移, 源指针变空 (Rust 的 move 同款语义) --\n");
30     {
31         auto owner = std::make_unique<Resource>(2);
32         printf(" move 前 owner=%p\n", (void *)owner.get());
33         auto thief = std::move(owner);
34         printf(" move 后 owner=%p (空) thief=%p\n", (void *)owner.get(),
35 (void *)thief.get());
36         if (!owner) printf(" 对空 unique_ptr 解引用是 UB, 只能先判空.\n");
37     }
38
39     printf("\n-- 传参把所有权交出去 --\n");
40     consume(std::make_unique<Resource>(3));
41
42     printf("\n-- 放进容器: 只移动, 指针稳定 --\n");
43     {
44         std::vector<std::unique_ptr<Resource>> pool;
45         pool.reserve(3);
46         for (int i = 4; i <= 6; ++i)
47             pool.push_back(std::make_unique<Resource>(i));
48         printf(" 容器里 %zu 个对象, 析构顺序与构造相反: \n", pool.size());
49     }
50
51     printf("\n-- 数组形式与自定义删除器 --\n");
```

```
50     {
51         auto arr = std::make_unique<int[]>(4);
52         arr[0] = 42;
53         printf("    unique_ptr<int[]> arr[0]=%d (析构用 delete[]) \n", arr[0]);
54         auto closer = std::unique_ptr<FILE, int (*)(FILE *)>
(fopen("/dev/null", "r"), &fclose);
55         printf("    自定义删除器: FILE* = %s\n", closer ? "已打开" : "打开失败");
56     }
57     printf("\n全部作用域结束, 所有资源都已释放.\n");
58     return 0;
59 }
```

```
1 // 例题 2-5: shared_ptr 的引用计数、循环引用, 以及 weak_ptr 怎么破坏
2 #include <cstdio>
3 #include <memory>
4
5 struct Node {
6     int id;
7     std::shared_ptr<Node> next;    // 强引用: 会推高计数
8     std::weak_ptr<Node> back;     // 弱引用: 不增加计数
9     explicit Node(int i) : id(i) { printf(" 构造 Node(%d)\n", id); }
10    ~Node() { printf(" 析构 Node(%d)\n", id); }
11 };
12
13 static void countDemo() {
14     printf("[引用计数变化]\n");
15     auto a = std::make_shared<Node>(1);
16     printf(" make_shared 后 use_count=%ld\n", a.use_count());
17     {
18         auto b = a;
19         printf(" 拷贝一份后 use_count=%ld\n", a.use_count());
20     }
21     printf(" 副本销毁后 use_count=%ld\n", a.use_count());
22     auto c = std::move(a);
23     printf(" move 之后: 源=%s, 新=%ld\n", a ? "非空" : "空", c.use_count());
24 }
25
26 static void cycleLeak() {
27     printf("\n[循环引用: 两个 shared_ptr 互指 → 谁都释放不了]\n");
28     auto x = std::make_shared<Node>(2);
29     auto y = std::make_shared<Node>(3);
30     x->next = y;
31     y->next = x;
32     printf(" 离开作用域前: use_count x=%ld y=%ld\n", x.use_count(),
33            y.use_count());
34     printf(" → 你会看到下面没有析构输出, 说明对象泄漏了 (ASan 能抓到) \n");
35 }
36
37 static void fixedCycle() {
38     printf("\n[用 weak_ptr 破坏: back 只是观察者]\n");
39     auto x = std::make_shared<Node>(4);
40     auto y = std::make_shared<Node>(5);
41     x->next = y;
42     y->back = x; // 弱引用不推高计数
43     printf(" 离开作用域前: x.use_count=%ld y.use_count=%ld\n", x.use_count(),
44            y.use_count());
45     if (auto locked = y->back.lock()) printf(" 弱引用能临时提升为强引用, 看到 Node(%d)\n", locked->id);
46 }
47
48 int main() {
49     countDemo();
50     cycleLeak();
51 }
```

```
49 | printf("    (上面没有 Node(2)/Node(3) 的析构 — 计数永远不归零) \n");
50 | fixedCycle();
51 | printf("\nNode(4)/Node(5) 会在离开作用域时正常析构。 \n");
52 | return 0;
53 | }
```

```
1 // 例题 3-1: 多继承的内存布局 —— 一个对象里为什么会有两个 vptr
2 #include <cstdio>
3
4 struct A {
5     int a = 1;
6     virtual void fa() { printf("A::fa()\n"); }
7     virtual ~A() = default;
8 };
9
10 struct B {
11     int b = 2;
12     virtual void fb() { printf("B::fb()\n"); }
13     virtual ~B() = default;
14 };
15
16 struct C : A, B {
17     int c = 3;
18     void fa() override { printf("C::fa() (重写 A 的) \n"); }
19     void fb() override { printf("C::fb() (重写 B 的) \n"); }
20     ~C() override = default;
21 };
22
23 int main() {
24     printf("sizeof(A)=%zu sizeof(B)=%zu sizeof(C)=%zu\n", sizeof(A),
25         sizeof(B), sizeof(C));
26     printf("→ C = A 子对象(vptr+int, 16) + B 子对象(vptr+int, 16) + int c + 填
27         充\n\n");
28
29     C obj;
30     A *pa = &obj;
31     B *pb = &obj;
32     C *pc = &obj;
33     printf("同一个对象, 三个指针指向不同的位置: \n");
34     printf("  pc = %p (对象首地址) \n", (void *)pc);
35     printf("  pa = %p (A 子对象, 偏移 %ld) \n", (void *)pa, (long)((char *)pa -
36         (char *)pc));
37     printf("  pb = %p (B 子对象, 偏移 %ld) \n", (void *)pb, (long)((char *)pb -
38         (char *)pc));
39
40     printf("\n两个子对象各有一张虚表: \n");
41     printf("  A 子对象的 vptr = %p\n", *reinterpret_cast<void **>(pa));
42     printf("  B 子对象的 vptr = %p\n", *reinterpret_cast<void **>(pb));
43
44     printf("\n通过不同指针调用, 都是虚调用, 各自查各自的表: \n");
45     pa->fa();
46     pb->fb();
47
48     printf("\n—— 指针调整是编译器做的, 不是免费的 ——\n");
49     B *fromC = static_cast<B *>(pc); // 正确: C 明确继承 B, 编译器自动偏移
50     B *wrong = reinterpret_cast<B *>(pa); // 危险: 按字节重解释, 地址没有偏移
51     B *cross = dynamic_cast<B *>(pa); // 运行时跨类转换 (A 多态才允许)
```

```
48     printf("  static_cast<B*>(pc)      = %p (正确, 从对象首地址偏移到 B 子对象)
    \n", (void *)fromC);
49     printf("  reinterpret_cast<B*>(pa) = %p (错, 等于把 A 子对象当 B 用 → UB)
    \n", (void *)wrong);
50     printf("  dynamic_cast<B*>(pa)      = %p (正确, 代价是运行时检查) \n", (void
    *)cross);
51
52     printf("\n-- 用基类指针 delete: 虚析构保证完整析构 --\n");
53     A *heap = new C();
54     printf("  delete 前先看 pb 偏移: B* p = %p\n", (void *)dynamic_cast<B *>
    (heap));
55     delete heap;
56     return 0;
57 }
```

```
1 // 例题 3-2: 菱形继承与虚继承 —— 两份 Base 还是一份 Base
2 #include <cstdio>
3
4 struct Base {
5     int value = 10;
6     virtual void show() { printf("Base::show() value=%d\n", value); }
7     virtual ~Base() = default;
8 };
9
10 // 不用虚继承: D 里有两份 Base
11 struct Left : Base {};
12 struct Right : Base {};
13 struct Diamond : Left, Right {};
14
15 // 用虚继承: 只剩一份 Base, 编译器插入 vbptr/偏移表来定位它
16 struct VLeft : virtual Base {};
17 struct VRight : virtual Base {};
18 struct VShape : VLeft, VRight {
19     void show() override { printf("VShape::show() value=%d (唯一一份 Base)\n", value); }
20 };
21
22 int main() {
23     printf("sizeof(Base)=%zu sizeof(Left)=%zu sizeof(Diamond)=%zu\n",
24           sizeof(Base), sizeof(Left), sizeof(Diamond));
25     printf("→ Diamond 里有两份 Base (2 个 vptr + 2 个 int) \n");
26     printf("sizeof(VLeft)=%zu sizeof(VRight)=%zu sizeof(VShape)=%zu\n",
27           sizeof(VLeft), sizeof(VRight), sizeof(VShape));
28     printf("→ 虚继承后只有一份 Base, 多出来的是指向公共 Base 的 vbptr\n\n");
29
30     Diamond d;
31     printf("[非虚继承]\n");
32     d.Left::value = 1;
33     d.Right::value = 2;
34     printf(" d.Left::value=%d 落在 %p\n", d.Left::value, (void *)&d.Left::value);
35     printf(" d.Right::value=%d 落在 %p\n", d.Right::value, (void *)&d.Right::value);
36     printf(" 两个 Base 子对象地址相差 %ld 字节, 写 d.value 会报二义性编译错误.\n\n",
37           (long)((char *)&d.Right::value - (char *)&d.Left::value));
38
39     VShape s;
40     printf("[虚继承]\n");
41     s.value = 99; // 只有一份, 不需要限定名
42     printf(" &s.value = %p, VLeft* 与 VRight* 指向的公共 Base 相同: %s\n",
43           (void *)&s.value,
44           (void *)static_cast<Base *>(static_cast<VLeft *>(&s)) ==
45           (void *)static_cast<Base *>(static_cast<VRight *>(&s))
46           ? "是"
47           : "否");
```

```
48     printf("    通过左路指针调用虚函数: \n");
49     static_cast<VLeft *>(&s)->show();
50
51     printf("\n[内存布局速查]\n");
52     printf("    非虚菱形: Diamond{ Left{ vptr, Base{...}? }, Right{...} } → 2 份
数据, 改一处另一处不动\n");
53     printf("    虚菱形:    VShape{ vbptr..., 公共 Base 一份 } → 只有一份数据, 代价是每
次访问多一次间接寻址\n");
54     return 0;
55 }
```

```
1 // 例题 3-3: 堆越界写 —— 编译器不拦, ASan 一抓一个准
2 // 编译: clang++ -std=c++17 -g -O0 -fsanitize=address p3_03_overflow.cpp -o p3_03
3 #include <cstdio>
4
5 int main() {
6     int *arr = new int[4];
7     for (int i = 0; i < 4; ++i) arr[i] = i;
8     printf("正常写完 4 个元素: arr[3]=%d\n", arr[3]);
9     printf("下面故意越界写 arr[4].....\n");
10    fflush(stdout);
11    arr[4] = 99; // ← 越界: ASan 会在这里报 heap-buffer-overflow
12    printf("这行几乎不会执行到: %d\n", arr[4]);
13    delete[] arr;
14    return 0;
15 }
```

```
1 // 例题 3-4: 释放后使用 (野指针) —— 内存已经还给分配器, 读到的可能是任意值
2 // 编译: clang++ -std=c++17 -g -O0 -fsanitize=address p3_04_use_after_free.cpp -o p3_04
3 #include <cstdio>
4
5 int main() {
6     int *p = new int(7);
7     printf("分配: p=%p 值=%d\n", (void *)p, *p);
8     delete p;
9     printf("已 delete p, 但指针变量还在 (这就是野指针) \n");
10    fflush(stdout);
11    printf("读野指针: *p=%d\n", *p);    // ← ASan 报 heap-use-after-free
12    p = nullptr;                        // 好习惯: 释放后置空
13    if (!p) printf("置空后可以安全判断: p == nullptr\n");
14    return 0;
15 }
```

```
1 // 例题 3-5: 重复释放 —— 同一块内存还两次, 堆元数据被破坏
2 // 编译: clang++ -std=c++17 -g -O0 -fsanitize=address p3_05_double_free.cpp -o p3_05
3 #include <cstdio>
4 #include <memory>
5
6 int main() {
7     printf("--- 正面: 所有权写进类型里, 就没有第二次释放的机会 --\n");
8     {
9         auto safe = std::make_unique<int>(2);
10        printf("    unique_ptr 管着 %d, 作用域结束自动释放一次\n", *safe);
11        // delete safe.get(); // 想重复释放得先绕过类型系统: unique_ptr 会再删一次
12    }
13    printf("    离开作用域, 资源恰好释放一次.\n\n");
14
15    printf("--- 反面: 裸指针 delete 两次 --\n");
16    int *raw = new int(1);
17    delete raw;
18    fflush(stdout);
19    delete raw; // ← ASan 报 attempting double-free
20    return 0;
21 }
```

```
1 // 例题 3-6: 综合项目骨架 —— 多态成绩管理 (vector + 智能指针 + 文件 + RAII)
2 // 每个设计点都对应前面某道例题: 多态=3-1、所有权=2-4、异常安全=2-3
3 #include <cstdio>
4 #include <fstream>
5 #include <memory>
6 #include <sstream>
7 #include <string>
8 #include <vector>
9
10 class Student {
11     public:
12         Student(std::string name, double base) : name_(std::move(name)),
13         base_(base) {
14             printf(" 构造 %-8s 对象 @ %p\n", name_.c_str(), (void *)this);
15         }
16         virtual ~Student() { printf(" 析构 %-8s @ %p\n", name_.c_str(), (void *)this); }
17
18         virtual double finalScore() const = 0;    // 纯虚: 算法由子类决定
19         virtual const char *kind() const = 0;
20         virtual std::string describe() const {    // 基类实现里调用虚函数 → 多态
21             char buffer[128];
22             snprintf(buffer, sizeof(buffer), "%-8s %-8s 卷面 %5.1f → 总评 %5.1f",
23                 name_.c_str(), kind(), base_, finalScore());
24             return buffer;
25         }
26         const std::string &name() const { return name_; }
27         double baseScore() const { return base_; }
28     protected:
29         std::string name_;
30         double base_;
31 };
32
33 class Regular : public Student {
34     public:
35         using Student::Student;
36         double finalScore() const override { return base_; }
37         const char *kind() const override { return "普通生"; }
38 };
39
40 class Honors : public Student {
41     public:
42         using Student::Student;
43         double finalScore() const override { return base_ * 1.15; }    // 加分政策
44         const char *kind() const override { return "优生"; }
45 };
46
47 class Makeup : public Student {
48     public:
49         using Student::Student;
```

```

50     double finalScore() const override { return base_ * 0.9; }      // 补考折算
51     const char *kind() const override { return "补考生"; }
52 };
53
54 class GradeBook {
55 public:
56     void add(std::unique_ptr<Student> student) {
57         students_.push_back(std::move(student)); }
58
59     void report() const {
60         printf("-- 名册 (按对象真实类型多态求值) --\n");
61         for (const auto &student : students_) {
62             printf("  %s\n", student->describe().c_str());
63         }
64         if (!students_.empty()) {
65             printf("  平均总评 = %.2f\n", average());
66             void **vptr = *reinterpret_cast<void ***>
(student_.front().get());
67             printf("  首个对象的 vptr = %p (基类指针 + 虚表 → 多态的机械本质) \n",
68             (void *)vptr);
69         }
70
71     double average() const {
72         if (students_.empty()) return 0.0;
73         double sum = 0.0;
74         for (const auto &student : students_) sum += student->finalScore();
75         return sum / static_cast<double>(students_.size());
76     }
77
78     bool save(const std::string &path) const {
79         std::ofstream out(path);          // RAll: 离开函数自动关闭
80         if (!out) return false;
81         for (const auto &student : students_) {
82             out << student->kind() << ' ' << student->name() << ' ' <<
student->baseScore() << '\n';
83         }
84         return static_cast<bool>(out);
85     }
86
87     bool load(const std::string &path) {
88         std::ifstream in(path);
89         if (!in) return false;
90         students_.clear();
91         std::string kind, name;
92         double base = 0.0;
93         while (in >> kind >> name >> base) {
94             if (kind == "优等生") add(std::make_unique<Honors>(name, base));
95             else if (kind == "补考生") add(std::make_unique<Makeup>(name,
base));
96             else add(std::make_unique<Regular>(name, base));
97         }
98         return true;
99     }

```

```

99
100     private:
101         std::vector<std::unique_ptr<Student>> students_; // 独占所有权，容器只移动
102     };
103
104     int main() {
105         printf("sizeof(Regular)=%zu sizeof(Honors)=%zu sizeof(Makeup)=%zu (都含
106             vptr) \n\n",
107             sizeof(Regular), sizeof(Honors), sizeof(Makeup));
108
109         const std::string path = "/tmp/cpp_gradebook.txt";
110         {
111             GradeBook book;
112             printf("-- 构建名册 --\n");
113             book.add(std::make_unique<Honors>("Tom", 85));
114             book.add(std::make_unique<Regular>("Jerry", 78));
115             book.add(std::make_unique<Makeup>("Spike", 60));
116             printf("\n");
117             book.report();
118             printf("\n-- 持久化 (ofstream RAI, 无需手动 close) --\n");
119             printf("    保存到 %s: %s\n", path.c_str(), book.save(path) ? "成功" :
120                 "失败");
121             printf("\n-- 读回来 (工厂 + unique_ptr 重新建对象) --\n");
122             GradeBook reloaded;
123             if (reloaded.load(path)) {
124                 printf("\n");
125                 reloaded.report();
126             }
127             // ← 两个 GradeBook 在这里析构: vector 析构 → unique_ptr 析构 → 虚析构 → 完整释放
128             printf("\n所有对象已按构造的逆序释放, 没有裸 new/delete, 也没有泄漏.\n");
129             return 0;
130         }
131     }

```

```

1 // 例题 3-7: 不用调试器, 靠"打印地址 + 看字节"把问题缩小到一行
2 // 三个现场: 对象切片、悬空引用、容器扩容后旧引用失效
3 #include <cstdio>
4 #include <cstring>
5 #include <memory>
6 #include <string>
7 #include <vector>
8
9 static void hexdump(const void *data, std::size_t bytes, const char *label)
10 {
11     const unsigned char *p = static_cast<const unsigned char *>(data);
12     printf("%-16s @%p : ", label, data);
13     for (std::size_t i = 0; i < bytes; ++i) printf("%02x ", p[i]);
14     printf("\n");
15 }
16 // ----- 现场 1: 对象切片 (不崩, 但行为悄悄变了) -----
17 struct Base {
18     int id;
19     explicit Base(int i) : id(i) {}
20     virtual const char *who() const { return "Base"; }
21     virtual ~Base() = default;
22 };
23
24 struct Derived : Base {
25     char extra; // 子类新增的成员
26     explicit Derived(int i) : Base(i), extra('D') {}
27     const char *who() const override { return "Derived"; }
28 };
29
30 static void byValue(Base b) { // ← 按值传: 只复制 Base 那部分
31     printf(" byValue 收到的对象: sizeof=%zu vptr=%p who()=%s\n",
32           sizeof(b), *reinterpret_cast<void **>(&b), b.who());
33 }
34
35 static void byRef(Base &b) {
36     printf(" byRef 收到的对象: sizeof=%zu vptr=%p who()=%s\n",
37           sizeof(b), *reinterpret_cast<void **>(&b), b.who());
38 }
39
40 int main() {
41     printf("=== 现场 1: 对象切片 ===\n");
42     Derived d(7);
43     printf(" 原始对象: sizeof=%zu vptr=%p\n", sizeof(d),
44           *reinterpret_cast<void **>(&d));
45     hexdump(&d, sizeof(d), "Derived 原始");
46     byRef(d); // 引用: 对象没被复制, 虚调用仍然正确
47     byValue(d); // 值传递: Derived 被"切成"Base, extra 丢了、虚表也换成 Base 的
48     printf(" → 定位手法: 打印 vptr. 两个 vptr 不同, 就说明对象被切片了.\n");
49     printf(" → 修法: 接口一律按引用/指针传递, 或显式 clone 成 unique_ptr.\n\n");

```

```

50     printf("=== 现场 2: 悬空引用 ===\n");
51     const std::string *dangling = nullptr;
52     {
53         std::string local = "临时字符串";
54         dangling = &local;
55         printf("  作用域内 local @%p 内容=\"%s\"\n", (const void *)&local,
local.c_str());
56     }
57     printf("  local 已销毁, 但我们还拿着地址 %p\n", (const void *)dangling);
58     printf("  → 继续解引用就是 UB: 内容可能看起来是对的, 也可能乱码, 随机性正是它可怕的地方\n");
59     printf("  → 修法: 用 shared_ptr/unique_ptr 表达所有权, 或复制一份数据\n\n");
60
61     printf("=== 现场 3: 容器扩容后旧引用失效 ===\n");
62     std::vector<std::string> names{"a", "b"};
63     const std::string &first = names.front();
64     printf("  扩容前 names.data()=%p, first 引用地址 %p\n",
        (const void *)names.data(), (const void *)&first);
65     for (int i = 0; i < 20; ++i) names.push_back("x");
66     printf("  扩容后 names.data()=%p, capacity=%zu\n",
        (const void *)names.data(), names.capacity());
67     printf("  → 数据已经搬家, first 指向的是旧地址 (悬空) \n");
68     printf("  → 修法: 要么先 reserve 足够容量, 要么用下标/迭代器重新取\n\n");
69
70     printf("=== 排坑四步法 (把大问题切成一行) ===\n");
71     printf("  1. 复现: 写出 20 行以内的最小例子, 稳定触发\n");
72     printf("  2. 观测: 打印关键地址、sizeof、vptr、size/capacity、引用计数\n");
73     printf("  3. 对比: 正常路径 vs 异常路径的字节差异 (hexdump 对照) \n");
74     printf("  4. 验证: 用 ASan/UBSan 复跑, 确认修完之后不再报错\n");
75     return 0;
76 }
77
78

```